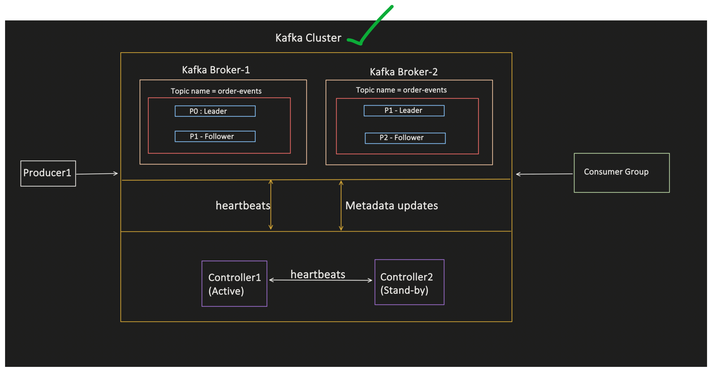
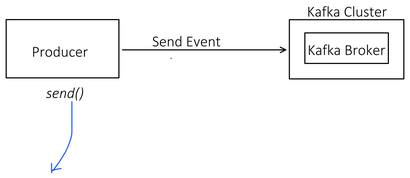
8-9. Producer Setup

Monday, 2 March 2026 12:20 PM

So we have already setup the Kafka Cluster:

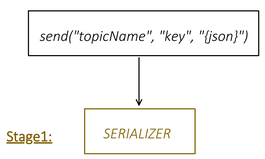


Before we proceed with Producer setup, lets understand the producer workflow:

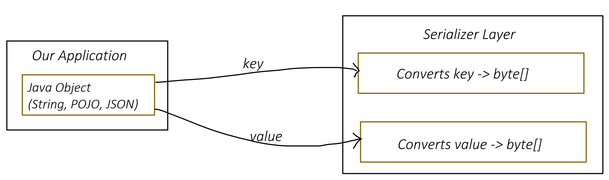


- When Producer try to send a message, it does not immediately send it to a broker.
- Instead message goes through multiple internal stages inside the producer, before any network call happens.

Let understand what steps are actually involved behind the scene:



- Our message (event) is generally a Java Object but Kafka understands only bytes. That's why Serializer is required.
- Kafka does not know, what is the data type of our message (Integer, String, Custom POJO, JSON etc.). That's why it want us to choose proper SERIALIZER.
- If Serialization stage fails, message will not move to next stage.



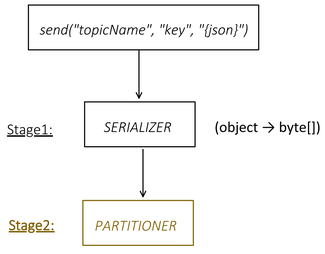
Every serializer implements this interface:

```
public interface Serializer<T> {
        byte[] serialize(String topic, T data);
}
```

There are many built-in Serializer, some of them are:

Serializer	Java Type
StringSerializer	String
IntegerSerializer	Integer
LongSerializer	Long
DoubleSerializer	Double
FloatSerializer	Float
ShortSerializer	Short
JsonSerializer	Any POJO (most useful, coverts java object to bytes)
Custom Serializer	When we want full control: - encrypt the data before byte conversion. - Removing sensitive fields before sending. etc.

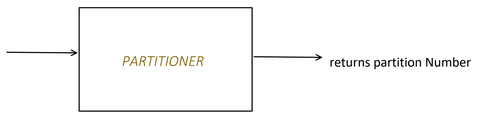
We will see this with an example, when we will setup the producer.



At this stage, producer decides "Which partition should this message go to?"

```
public interface Partitioner {

 int partition (
            String topic,
            Object key,
            byte[] keyBytes,
            Object value,
            byte[] valueBytes,
            Cluster cluster
     );
}
```



Case 1 : Partition explicitly provided

We can explicitly provide the partition no, so in this scenario, Partitioner does nothing.

KafkaTemplate.java

```
@Override
public CompletableFuture<SendResult<K, V>> send(String topic, Integer partition, K key, @Nullable V data) {
            ProducerRecord<K, V> producerRecord = new ProducerRecord<>(topic, partition, key, data);
            return observeSend(producerRecord);
}
```

Case 2 : Key is provided

Kafka ensures same key always goes to same partition.

KafkaTemplate.java

```
@Override
public CompletableFuture<SendResult<K, V>> send(String topic, K key, @Nullable V data) {
        ProducerRecord<K, V> producerRecord = new ProducerRecord<>(topic, key, data);
        return observeSend(producerRecord);
}
```

partition = hash(keyBytes) % numberOfPartitions

Case 3 : No Key is provided

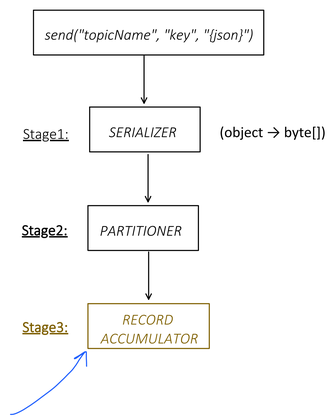
KafkaTemplate.java

```
@Override
public CompletableFuture<SendResult<K, V>> send(String topic, @Nullable V data) {
        ProducerRecord<K, V> producerRecord = new ProducerRecord<>(topic, data);
        return observeSend(producerRecord);
}
```

Pre Kafka 2.4: Round Robin Strategy

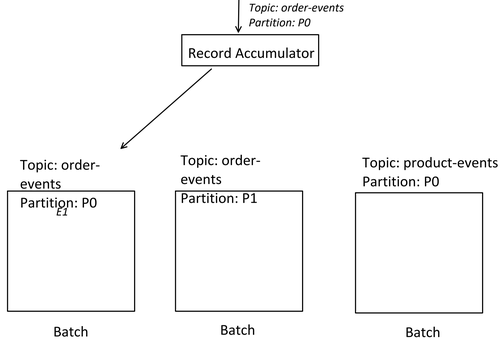
From Kafka 2.4 onwards: Sticky Partitioning Strategy

Whats the difference? Will explain after next stage



- In-memory buffer inside Kafka Producer, that collects events per topic-partition and group them into batches.

- And because of this only, Kaka Producer achieve very high throughput (ability to transmit large volume of data within a given timeframe)



So, now we knows that, Producer always maintain topic-partition wise batch, and it send batch to the broker (not individual events).

Now lets come back to the point:

Pre Kafka 2.4: Round Robin Partitioning Strategy

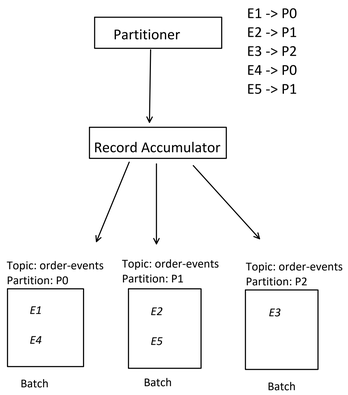
From Kafka 2.4 onwards: Sticky Partitioning Strategy

Example:

Topic: order-events, Key = null, Value = E1
Topic: order-events, Key = null, Value = E2
Topic: order-events, Key = null, Value = E3
Topic: order-events, Key = null, Value = E4
Topic: order-events, Key = null, Value = E5

Number of Partitions for order-events topic : [P0, P1, P2]

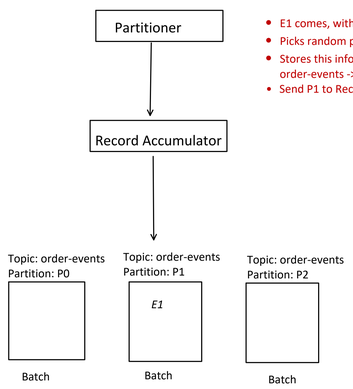
Round Robin Partitioning Strategy



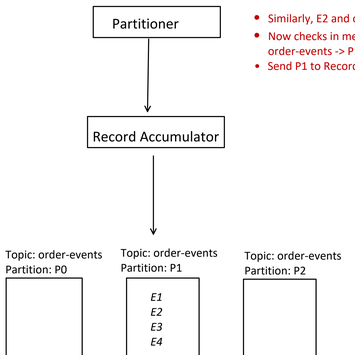
So now, these 3 batches will be send to broker.

Problem with this approach is: Batch size is very small. And therefore multiple network calls to Kafka Broker.

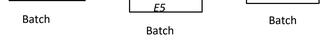
Sticky Partitioning Strategy



- E1 comes, with no key.
- Picks random partition of order-events topic, say P1
- Stores this info in-memory (Sticky):
  order-events -> P1
- Send P1 to Record Accumulator



- Similarly, E2 and other events comes with no key.
- Now checks in memory sticky info for order-events topic
  order-events -> P1
- Send P1 to Record Accumulator

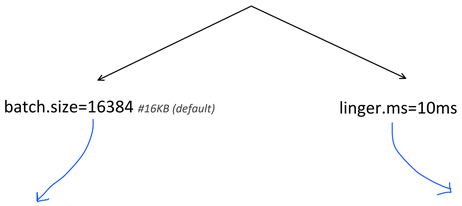


Advantage:

Events stick to the same partition **until the current batch is closed or a new batch needs to be created**..

So less number of batch and therefore less number of network calls.

Now question we might get is, "**How to control the batch?**"



- Means maximum memory allocated per partition batch.
- For example :
  if 1 Event size = 1KB then
  Batch will hold ~16 messages.

  and when batch size is reached to its limit, batch is closed and its reading for sending.

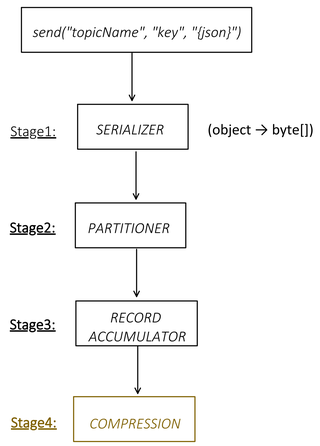
- Maximum time producer will wait for more records to fill the batch.
- This make sure that, producer do not wait indefinitely for a batch to fill when traffic is low.

  For example:

  Batch size = 16KB
  Event size = 1KB

  means 1 batch can hold 16 messages. But traffic is less and only 5 events yet added in batch.

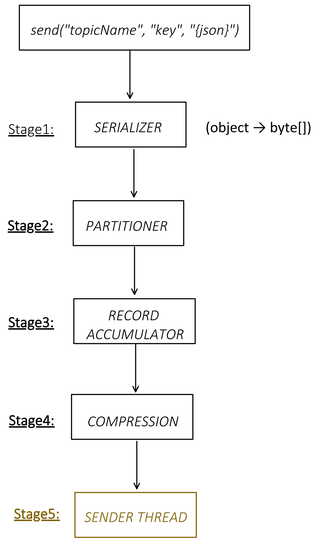
  Producer will max wait 10ms for more events, else after 10ms it close the batch and it will be ready to send.



- Batches which are ready to send are compressed to reduce network usage and thus achieve higher throughput.
  Ex:
  Batch before compression = 64KB
  After compression = 18KB

- Kafka stores these compressed batches thus save disk space too.

When we will setup the producer, will show how in properties we can add this compression.



- Application thread never send data to Kafka broker.
- Background sender thread does the actual communication.

Application thread work is till writing the record to Record Accumulator, after that sender thread comes into the picture:

Sender thread Responsibility:

1. check RecordAccumulator for ready batches
2. Group batches by broker leader. Ex:
   i. Batch for TopicA-Partition0 -> Leader is Broker1
   ii. Batch for TopicA-Parition5 -> Leader is Broker1
   iii. Batch for TopicB-Partition1 -> Leader is Broker1

   So, broker leader wise it will group the batches and wrap it up in 1 ProduceRequest and send it in 1 network call.
3. send request using NetworkClient
4. process responses (acks, retries)
5. Handle retries if needed
6. Complete future and invoke callbacks.

Now question might comes to you, can we control these Sender Thread like how many they should exist etc.

Answer is NO, we don't have a way to control this. Kafka manage it internally.

But wait, there is 1 thing which we can control :

Say we have 3 Brokers, so Sender Thread will create 3 TCP connection (one for each) to send the message to the respective broker.

max.in.flight.requests.per.connection = 5 (default)

This parameter controls the maximum number of unacknowledged requests the client will send on a single connection before blocking.

So we have 3 TCP connection, so max 15 requests can be in-flight across the cluster at once.

But very important:

max.in.flight.requests.per.connection > 1
and
Retries is also > 0

Then there is risk of message reordering.

Ex:

Request A -> in-flight
Request B -> in-flight

Request A -> fails
Request B -> success (written successfully in partition)

Request A -> retries and Success (written after Request B)

This Risk is resolved through IDEMPTOENCY set to TRUE, which we will see separately.

Now, we know a bit about Producer, lets start our Producer Setup:

Step1: Dependency

pom.xml

```
<dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
</dependency>
```

This single dependency pulls:

spring-kafka = **Spring wrapper layer + Kafka client dependency**.

Spring wrapper layer:

- KafkaTemplate - which is required to send messages
- KafkaAdmin - which is required to manage topic

etc. These are Spring abstraction build on top of Apache kafka client.

And it also bring actual Kafka client library. The real implementation which KafkaTemplate, KafkaAdmin etc. uses.

Step2: Add Bootstrap Brokers

```
server.port=8081
spring.application.name=kafka-producer-service

#Both brokers: if one is down, producer connects to the other
spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

#Metadata refresh interval, default is 5minutes only
spring.kafka.producer.properties.metadata.max.age.ms=300000
```

This bootstrap info is required because say initially producer do not know, which broker is leader for which partition or for topic creation it can not directly talk with controller, so it need 1 starting communication point.

- If it's a topic creation request, that broker will pass the request to active controller.

- *If its regarding cluster metadata fetch, if broker has the info it will send else it will fetch from controller and send it back to producer.*
- *Producer will store it in-memory and keep refresh it after 5 minutes (default) or if any issue comes during connection with the leader of the partition.*

Step3: Topic Creation

```
@Configuration
public class KafkaProducerConfig {

   @Bean
   public NewTopic orderEventsTopic() {

         return TopicBuilder.name("order-events").build();
   }
}
```

We have seen in Cluster setup part that, if during topic creation, partition and replication factor information is not provided, then from controller properties default values will be picked.

Similarly we can provide default value of Segment size, clean up policy in Broker properties.

Same config we can also provide during Topic creation too, TopicBuilder provide us a way to set the configurations, we have so many configuration, showing some important configuration below:

```
@Bean
   public NewTopic orderEventsTopic() {

      return TopicBuilder.name("order-events")

            //if not provided here, will be picked from controller properties
            .partitions(3)
            .replicas(2)

            // if not provided here, will be picked from broker properties
           .config(TopicConfig.RETENTION_MS_CONFIG, "604800000")

            // if not provided here, will be picked from broker properties
           .config(TopicConfig.CLEANUP_POLICY_CONFIG, "delete")

            // if not provided here, will be picked from broker properties
           .config(TopicConfig.MIN_IN_SYNC_REPLICAS_CONFIG, "2")

            // if not provided here, will be picked from broker properties
           .config(TopicConfig.SEGMENT_BYTES_CONFIG, "1073741824")
            .build();
}
```

controller1.properties

```
process.roles=controller

node.id=1


listeners=CONTROLLER://:9093

controller.listener.names=CONTROLLER

controller.quorum.voters=1@localhost:9093,2@localhost:9193


# PLAINTEXT (no encryption). In production, we should use SSL.
listener.security.protocol.map=CONTROLLER:PLAINTEXT

log.dirs=/tmp/controller1-logs

# Default partitions when creating a topic without specifying
num.partitions=3

default.replication.factor=2
```

broker1.properties

```
# WHERE to fetch metadata from (which controllers to connect to)
controller.quorum.voters=1@localhost:9093,2@localhost:9193


# This is where our Spring Boot apps(producer/consumer) will connect
listeners=BROKER://:9092

# passed this address to controller to store in their cluster metadata
advertised.listeners=BROKER://localhost:9092

# internally when follower broker try to fetch from leader, what listener they need to use.
inter.broker.listener.name=BROKER

#  when broker try to communicate with controller what listener they need to use.

controller.listener.names=CONTROLLER

listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT

log.dirs=/tmp/broker1-logs

# Keep messages for 7 days (168 hours)
log.retention.hours=168

# Max size of a single log segment file (1GB)
log.segment.bytes=1073741824

# __consumer_offsets topic also replicate across both brokers
\offsets.topic.replication.factor=2
```

During application startup, it will check if specific topic exist or not, if not then it will send the create topic request.

Step4: Send Event

Usecase1:

- *SERIALIZER*:
  - Key : String
  - Value : JSON
- *PARTITIONER*:
  - Send with Key, so that related events goes to same partition.
- *RECORD ACCUMULATOR*
  - Max Batch size: 32KB
  - Linger = 20ms
  - Overall Buffer for batches = 32MB
  - When Overall Buffer is full, Send() message should wait for 60sec after that start throwing exception.
- *COMPRESSION*
  - *Use one out of many : snappy, gzip, lz4, zstd etc.*
- *SENDER THREAD*
  - *In-flight request per connection = 5*

- *3 times retry on transient failures*

OrderController.java

```java
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @Autowired
    OrderProducerService orderProducerService;

    @PostMapping("/with-key")
    public ResponseEntity<String> sendWithKey(@RequestBody Order order) {
            orderProducerService.sendWithKey(order);
            return ResponseEntity.accepted().body("Order created and event published!");
    }
}
```

OrderProducerService.java

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;
import java.util.concurrent.CompletableFuture;

@Service
public class OrderProducerService {

@Autowired
private KafkaTemplate<String, Order> kafkaTemplate;

public void sendWithKey(Order order) {
        String key = order.getOrderId();

        CompletableFuture<SendResult<String, Order>> future = kafkaTemplate.send("order-events", key, order);

        future.whenComplete((result, ex) -> {
                if (ex == null) {
                        System.out.println("Order event sent successfully");
                        System.out.println("Partition: " +  result.getRecordMetadata().partition());
                        System.out.println("Offset: " + result.getRecordMetadata().offset());
                } else {
                        System.out.println("Failed to send order event: " +  ex.getMessage());
                }
        });
    }
}
```

Order.java (POJO class)

```java
public class Order {
    private String orderId;
    private String customerId;
    private String productId;
    private Integer quantity;
    private Double totalAmount;
    private String status;

        //getters and setters
    }
```

application.properties

```
server.port=8081
spring.application.name=kafka-producer-service

# Both brokers listed for redundancy, if one is down, producer connects to the other
spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

# Metadata refresh interval: how often producer re-fetches cluster info (default 5 min)
spring.kafka.producer.properties.metadata.max.age.ms=300000

# Asking broker to reply only when all ISR successfully get the events
spring.kafka.producer.acks=all

#--------------SERIALIZER-------------
# Key needs to be String
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

# Value: JsonSerializer, auto-converts Java objects to JSON bytes
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer
```

```
#---------------RECORD ACCUMULATOR-------
# Batching, wait 20ms to collect more messages before sending
spring.kafka.producer.properties.linger.ms=20

# Max batch size, send when batch reaches 32KB (even if linger.ms hasn't expired)
spring.kafka.producer.properties.batch.size=32768

# Total buffer memory for all batches: 32MB (if full, send() will get blocked till space created or max block time)
spring.kafka.producer.properties.buffer.memory=33554432

# Max time send() blocks if buffer is full (default 60s), after this time, send() call will fail immediately
spring.kafka.producer.properties.max.block.ms=60000


#---------------COMPRESSION-------
# compresses entire batch before sending
spring.kafka.producer.properties.compression.type=snappy


#---------------SENDER THREAD-------
# Retry on transient failures (network timeout, leader change, etc.)
spring.kafka.producer.retries=3

# Max in-flight requests
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5
```

Usecase2:

- *SERIALIZER*:
  - Key : String
  - Value : Custom Serializer

- *PARTITIONER*:
  - Send without Key (Sticky Partitioning)

Rest all same like previous

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @Autowired
    OrderProducerService orderProducerService;

    @PostMapping("/with-no-key")
    public ResponseEntity<String> sendWithoutKey(@RequestBody Order order) {
            orderProducerService.sendWithoutKey(order);
            return ResponseEntity.accepted().body("Order created and event published!");
    }
}
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;
import java.util.concurrent.CompletableFuture;

@Service
public class OrderProducerService {

@Autowired
private KafkaTemplate<String, Order> kafkaTemplate;

public void sendWithoutKey(Order order) {
        String key = order.getOrderId();

        CompletableFuture<SendResult<String, Order>> future = kafkaTemplate.send("order-events", order);

        future.whenComplete((result, ex) -> {
                if (ex == null) {
                        System.out.println("Order event sent successfully");
                        System.out.println("Partition: " +  result.getRecordMetadata().partition());
                        System.out.println("Offset: " + result.getRecordMetadata().offset());
                } else {
                        System.out.println("Failed to send order event: " +  ex.getMessage());
                }
        });
```

```
    }
}
```

Custom Serializer:

```
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Serializer;
import java.util.LinkedHashMap;
import java.util.Map;

public class OrderSummarySerializer implements Serializer<Order> {

    private final ObjectMapper objectMapper;

    public OrderSummarySerializer() {
        this.objectMapper = new ObjectMapper();
    }

    @Override
    public byte[] serialize(String topic, Order order) {
        if (order == null) {
            return null;
        }

        try {
                // Build a map with only the fields we want to expose, rest I removed it
                Map<String, Object> summary = new LinkedHashMap<>();
                summary.put("orderId", order.getOrderId());
                summary.put("productId", order.getProductId());
                return objectMapper.writeValueAsBytes(summary);

        } catch (JsonProcessingException e) {
            throw new RuntimeException("Failed to serialize Order summary", e);
        }
    }
}
```

application.properties

```
server.port=8081
spring.application.name=kafka-producer-service

# Both brokers listed for redundancy, if one is down, producer connects to the other
spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

# Metadata refresh interval: how often producer re-fetches cluster info (default 5 min)
spring.kafka.producer.properties.metadata.max.age.ms=300000

# Asking broker to reply only when all ISR successfully get the events
spring.kafka.producer.acks=all

#--------------SERIALIZER------------
# Key needs to be String
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

# Value: custom serializer
spring.kafka.producer.value-serializer=com.eda.producer.serializer.OrderSummarySerializer

#--------------RECORD ACCUMULATOR-------
# Batching, wait 20ms to collect more messages before sending
spring.kafka.producer.properties.linger.ms=20

# Max batch size, send when batch reaches 32KB (even if linger.ms hasn't expired)
spring.kafka.producer.properties.batch.size=32768

# Total buffer memory for all batches: 32MB (if full, send() will get blocked till space created or max block
time)
spring.kafka.producer.properties.buffer.memory=33554432

# Max time send() blocks if buffer is full (default 60s), after this time, send() call will fail immediately
spring.kafka.producer.properties.max.block.ms=60000


#--------------COMPRESSION-------
# compresses entire batch before sending
spring.kafka.producer.properties.compression.type=snappy


#--------------SENDER THREAD-------
# Retry on transient failures (network timeout, leader change, etc.)
spring.kafka.producer.retries=3

# Max in-flight requests
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5
```

But what if we have many producer:

- Some need JsonSerializer
- Some need custom Serializer

Then how in 1 application.properties we will define?

We can define @Bean

```
@Bean
public KafkaTemplate<String, Order> customSerializerKafkaTemplate(KafkaProperties kafkaProperties) {

    // Start with all producer properties from application.properties
    Map<String, Object> props =
                                new HashMap<>(kafkaProperties.buildProducerProperties(null));

    // Override ONLY the value serializer - everything else stays the same
    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, OrderSummarySerializer.class);

    DefaultKafkaProducerFactory<String, Order> factory =
                                            new DefaultKafkaProducerFactory<>(props);

    return new KafkaTemplate<>(factory);
}
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;
import java.util.concurrent.CompletableFuture;

@Service
public class OrderProducerService {

@Autowired
private KafkaTemplate<String, Order> kafkaTemplate;

@Autowired
@Qualifier("customSerializerKafkaTemplate")
private KafkaTemplate<String, Order> customSerializerKafkaTemplate;

public void sendWithoutKey(Order order) {
        String key = order.getOrderId();

        CompletableFuture<SendResult<String, Order>> future = customSerializerKafkaTemplate.send("order-events", order);

        future.whenComplete((result, ex) -> {
                if (ex == null) {
                        System.out.println("Order event sent successfully");
                       System.out.println("Partition: " +  result.getRecordMetadata().partition());
                        System.out.println("Offset: " + result.getRecordMetadata().offset());
                } else {
                        System.out.println("Failed to send order event: " +  ex.getMessage());
                }
        });
  }
}
```

Uses values from application.properties

Pick value from Configuration @Bean

In application.properties we can keep the default "JsonSerializer"

```
server.port=8081
spring.application.name=kafka-producer-service

# Both brokers listed for redundancy, if one is down, producer connects to the other
spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

# Metadata refresh interval: how often producer re-fetches cluster info (default 5 min)
spring.kafka.producer.properties.metadata.max.age.ms=300000

# Asking broker to reply only when all ISR successfully get the events
spring.kafka.producer.acks=all

#--------------SERIALIZER------------
# Key needs to be String
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

# Value: JsonSerializer, auto-converts Java objects to JSON bytes
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer

#--------------RECORD ACCUMULATOR-------
```

```
# Batching, wait 20ms to collect more messages before sending
spring.kafka.producer.properties.linger.ms=20

# Max batch size, send when batch reaches 32KB (even if linger.ms hasn't expired)
spring.kafka.producer.properties.batch.size=32768

# Total buffer memory for all batches: 32MB (if full, send() will get blocked till space created or max block time)
spring.kafka.producer.properties.buffer.memory=33554432

# Max time send() blocks if buffer is full (default 60s), after this time, send() call will fail immediately
spring.kafka.producer.properties.max.block.ms=60000


#---------------COMPRESSION-------
# compresses entire batch before sending
spring.kafka.producer.properties.compression.type=snappy


#---------------SENDER THREAD-------
# Retry on transient failures (network timeout, leader change, etc.)
spring.kafka.producer.retries=3

# Max in-flight requests
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5
```

Now, last part, we discussed that, when:

max.in.flight.requests.per.connection > 1
and
Retries is also > 0

Then there is risk of message reordering.

Ex:

Request A -> in-flight
Request B -> in-flight

Request A -> fails
Request B -> success (written successfully in partition)
Request A -> retries and Success (written after Request B)

How to resolve this Risk?